
LAB 3: VIRTUAL REALITY

VIRTUAL REALITY, USABILITY AND UNITY

GENERAL INFORMATION

This lab will focus on Virtual Reality (VR). For this lab, an application was created called “A Terrible Day in the Office”, which holds 5 mini-tasks that each have some bad usability practices built into them intentionally. The overall goal is to reduce the total time spent in performing each task.

Each mini-task is assigned to one group-member, and each group-member should make a change to their mini-task based on identified violations of the usability principles. This manual includes examples of changes that improve certain usability principles in order to engage students in creative thinking.

Before continuing to the rest of document, please ensure you have followed the steps outlined in Lab 3 – VR – Installation Guide.pdf in order to set up Unity and the VR application. For the second labs on VR, you should prepare for the lab by viewing the demo videos (available on canvas) of the VR task allocated to you and filling in the corresponding usability table. Make sure that one member of your group will upload one document to the assignment on canvas with all tables filled in. In addition, after carefully reading this manual, you should build your apk that enhances the usability principles of the “A Terrible Day in the Office” VR application and bring it to the lab to test it with the VR headset.

Don't forget to bring your student ID!

Outline of the document:

- General rules (i.e. what to *never* change)
- Task 1: Drawers
- Task 2: Coffee
- Task 3: Trash
- Task 4: Sponge
- Task 5: Contextual menu
- Deployment
- Casting

GENERAL RULES

In the application, many variables can be changed, but some things should not be changed to prevent the “cheating” of the tasks.

Non-changeables:

- Do not change the desk and floor positions and scales. Do not change the Room itself (see hierarchy)
 - Do not change the debug logs for the analysis
 - Do not change the spawn points
 - Do not change input bindings
 - Do not change general physics engine (e.g. FixedTimestep, gravity, etc.)
 - Do not replace models/meshes for the items used in the lab
 - Do not adapt the required number of x items in a task. For example, the Drawers task includes 4 files to be added per drawer. You should not change this amount. The same applies for all other tasks.
 - Do not let objects move automatically.
 - Do not rescale objects to make the task easier, e.g. you cannot make the sponge extremely big, whereas rescaling text/buttons in the UI is fine.
-

CLEANING TASK

In this task, the player needs to let the sponge touch various zones to “clean” them. The sponge is represented as a rectangular rigidbody with a collider (on children) and an XR grab interactable. The cleaning zones are represented as trigger colliders. There are 5 trigger colliders that should be touched with the sponge by the user.

Relevant Files in Hierarchy:

Parent: CleaningTask (attached: CleaningTaskController script)

- Sponge (RigidBody, XRGrabInteractable, attached: CleaningTask script)
 - o GreenPart (Collider and Material)
 - o YellowPart (Collider and Material)
- CleanColliders (5x trigger colliders)
- X – SpongeSpawnPoint (do not change)

Relevant Scripts:

- CleaningTaskController (do not change)
 - o Handles the reset of CleaningTask (just resetting location of the sponge, not the progress)
- CleaningTask (can be changed)
 - o Fields (do not change or remove the current, you can add more)
 - targets (Collider[]) (List of colliders that represent the cleaning zones. Assign in inspector)
 - spongeRigidbody (Rigidbody) (Assign in inspector, this triggers the cleaning)
 - cleaningTask (bool) (internal state, becomes true when all zones are cleaned)
 - IsComplete (bool, read-only) (Public property that returns whether the task is complete.)
 - o Methods:
 - Start() do not change (initializes everything)
 - **OnTriggerEnter()** can be changed
Called whenever the sponge touches a zone. Marks zones as cleaned and checks for completion.

Example change

Add a sound when the player touches a cleaning zone with the sponge.

- Find the CleaningTask script, and the method on OnTriggerEnter()
- Now, find the line Debug.Log(“CleaningSpot touched”)
- Above the Debug.Log command, add GetComponent<AudioSource>().Play();
- In the Unity editor, select the Sponge object in the hierarchy.
- Add an Audio Source component to it.
- Drag one of the provided audio files (found in the Audio folder) into the AudioClip slot of the Audio Source.

DRAWER TASK

In this task, the player has to correctly file 8 files in the 2 drawers. The files are generated in a stack on the side of the desk, and can be respawned when needed. It's a bit unclear from the start where each file needs to be put.

Relevant files in the hierarchy:

Parent: DrawerTask (attached: Drawer Task Reset Controller)

- DrawerRight/DrawerLeft (prefab) (attached: DrawerTask)
 - o Frame
 - The outsides of the drawer, incl the physical stoppers of the drawer.
 - o Drawer_Itself
 - The insides of the drawer, including the “box” that is pulled out, the knob, and the socket interactors (snap locations the files can snap into)
- X – FileStack (attached: FileStackSpawner)
 - o Object holding a transform (where the files will spawn) and a connected script holding the logic for spawning files

Relevant scripts in the hierarchy

- DrawerResetController (do not change)
 - o Handles the reset of the drawers
- DrawerTask, per drawer (can be changed)
 - o Fields
 - FileType expected (which of the 2 files we need here)
 - XRSocketInteractor[] sockets (drag the sockets here in inspector)
 - RequiredCount =4 (the number of sockets that need to be filled)
 - Total-inserts/Wrong-Inserts/ErrorRate/IsComplete
 - o Methods
 - OnEnable (do not change) Adds listeners to the sockets
 - OnDisable (do not change) Remove listeners from the sockets
 - **OnSocketEntered** -> called when something entered a socket
 - **OnSocketExited** -> called when something exited a socket
 - **Recompute** -> called when OnSocketEntered and OnSocketExited
 - ResetState (do not change) Called by DrawerResetController
 - GetErrorMetrics (do not change) Calculates the error rate

Minimal change:

- In the inspector, you can change the fronts of the drawers to be the same color as the file that it needs to accept. In the hierarchy, go to DrawerTask, go to either Drawer and check what filetype this one accepts (see the Drawer Task Script Fields in the inspector) Now from the _Lab4Assets folder, drag the corresponding file-material (FileMaterialGreen or FileMaterialLightGreen) to the front of the drawer. Now it should be easier for the user to see which file goes where, of course, this is not a great solution, any ways how we can make this better?

COFFEE TASK

In this task, the player has to pour coffee in their coffee cup. The coffee pot has coffee particles coming out of the sprout tip, when this coffee is held at the correct angle. The coffee cup gets “filled” when the coffee sprout tip – ray is pointing at the cup for a long-enough time.

Relevant files in hierarchy: CoffeeTask

- CoffeePot (RigidBody, Collider, XR Grab Interactable)
 - o SproutTip (transform)
 - CoffeeParticles (particle system)
- EmptyCup (empty parent)
 - o Cup itself (RigidBody, Collider, XR Grab Interactable)
 - CupEnterLocation (Trigger Collider)
- X - SpawnPoints

Relevant scripts in the hierarchy:

CoffeeTask

- Fields
 - o XR Grab Interactables (Coffee and Cup)
 - o Spawn Points (CoffeePot and Cup transforms)
 - o Pour setup (spoutTip, particle system, collider at cup opening and raydistance (which is the length of the stream))
 - o On and Off Angle for the coffee pot
 - o Completion parameters
- Methods
 - o Update (can be changed) Runs pour detection, particle control, raycast hit check, and completion timing
 - o ResetTask (do not change) Drops held objects, stops/clears particles, resets state, respawns to spawn points, zeros rigidbodies
 - o StreamDir (do not change) Returns the forward direction of spoutTip used as the stream direction
 - o RayHitsCupMouth (can be changed) Raycasts from spoutTip along the stream to detect the cup mouth trigger
 - o SetParticles (can be changed) Starts or stops the pour particle system
 - o ForceRelease (can be changed) Cleanly exits all selections on a grabbed object
 - o ZeroBody (do not change) Clears linear and angular velocity and sleeps the rigidbody

Minimal change:

- A way to make this task more aligned with the usability principles, is by making the pouring angle more realistic. In the inspector, go to the CoffeeTask and in the inspector you can see the fields of the CoffeeTask script. Here, you can adapt the Angle On Deg and Angle Off Deg variables.

TRASH TASK

In this task, the player throws items of trash in the bin. When these are thrown from far enough away and with enough velocity, this will be counted as a “score”.

Relevant files in hierarchy: TrashTask

Parent file: TrashTask (attached: Trash Respawner)

- Bin (RigidBody, Colliders, ScoreZone and RimCenter included)
- Can (RigidBody, Collider, XR Grab Interactable, TrashItemThrowData script)
- Bottle (RigidBody, Collider, XR Grab Interactable, TrashItemThrowData script)
- X - Spawnpoints

Relevant scripts in the hierarchy:

- TrashBinScorer:
 - o Fields:
 - RimCenter (used to calculate direction of throw)
 - ScoreZone (trigger collider)
 - o Methods:
 - OnTriggerEnter: called when something enters the bin
 - OnTriggerExit: called when something is taken out of the bin
 - UpdateCompletion: do not change
- TrashItemThrowData: saves information of the throw with the OnRelease method
- TrashRespawner: do not change this, respawns the trash items

Minimal change:

- Indicate in the GUI what objects are trash and how many need to enter. You can decipher this by looking at the hierarchy, what objects are under the trash task and do they have the “TrashItemData” script attached? For this, in the hierarchy, go to the UI, Introscreen, then to the UI Root, then to the MainText, then you can find in the inspector the Text Mesh Pro - Text Input location with the instruction texts, adapt the text there.

CONTEXTUAL MENU TASK

In this task you want to make the contextual menu better. Right now it gives you some sort of description of the task, but its not very clear, it does not track progress, the buttons overlap etc.

Relevant files in hierarchy:

Parent is UI

- IntroScreen
 - X – Interaction Affordance (do not change)
 - UI Root
 - Non-changeables (background, outline etc)
 - MainText
 - Buttons
 - DrawerResetButton
 - CleaningResetButton
 - CoffeeResetButton
 - TrashResetButton
 - StartButton
 - QuitButton
 - In all of these ^ you can find “Button” in the Inspector, which has a block for “On Click” which then highlights what method is called when this is clicked.

And: GameManager with various scripts attached

Relevant Scripts

The buttons call their respective task’s reset button, or the Start and Quit in the GameController script, which uses ProgressTracker and RunSummaryAndQuit as well. Depending on what you want to change, you can use various methods/fields from these scripts.

Minimal change:

- You can drag the various reset buttons further apart in the editor, or make them smaller/bigger using the editor. You can explore the various tools in the scene-view, that change different properties of the object:



DEPLOYMENT

Testing without the VR headset is possible using the Simulator (see installation guide).

The VR headset is essentially an Android device and for this reason accepts apk files. A typical user would normally download a new app from the built-in store (similar to the Play Store on your phone). This process requires apps to be published and verified by Google (or by Meta on VR). While your app may soon be close to perfection in terms of the IUI requirements set in this course, it is not ready to be published yet, especially during development and debugging.

To work around this, we “sideload” the app directly from Unity to the VR headset, without going through the store. In Unity, go to **File > Build Profiles** and select **Android** under Platforms. In the Platform settings, under Run Device, you should now be able to select your VR headset. Click **Switch Platform** in the top right. This will trigger a recompilation of the project, so wait until it is complete.

If you have the VR headset:

- Connect your VR set to your laptop using the provided USB-C cable (you cannot use the original charging cable for this). Put the VR headset on, go to the notifications, and allow USB data transfer.
- In the Scene List, make sure that only Scenes/terribleOffice is selected.
- Ensure that the “Run Device” in Platform Settings is set to the serial number of the VR headset.
- Then click **Build and Run**. (If you get a pop-up about a project-id, you can press “yes”) Unity will ask you to choose a location to save your apk file, and the app will be built and sideloaded onto your VR headset.
- When it says it “succeeded” you can start the VR and start the application with the unity logo called “VR_Build”

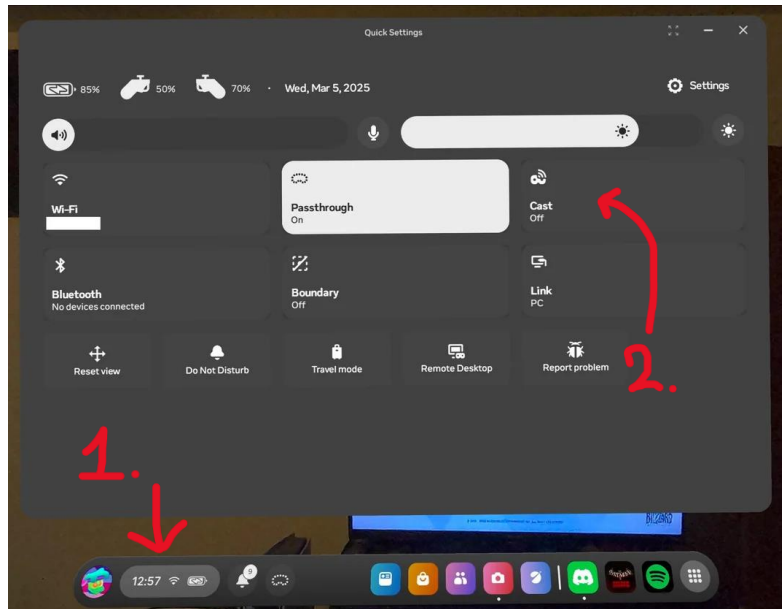
If you do not have the VR headset yet:

- You can already build .apk files to sideload later, as sometimes building can take a long time. Instead of doing Build and Run, we will run **Build** instead (If you get a pop-up about a project-id, you can press “yes”). This way you save the .apk file, but don't automatically load it onto the vr headset.
- Then, when you want to sideload onto the headset, go to the folder your build is located in, in the terminal. **Then if you get access to the VR headset:**
- Run: “adb uninstall com.unity.template.vr”
- And then run: “adb install -r your-build-name.apk”
- When it says “succeeded” you can start the VR and start the application with the unity logo called “VR_Build”

CASTING

Casting the view from your VR headset enables others to see what you are doing. Meta makes it possible to cast from the VR headset to either the app or from a browser, we will use the latter.

1. Connect the VR headset to the internet. The TAs will share the eduroam guest account to be used.
2. Ensure the computer you are casting to is also connected to eduroam.
3. In the VR headset, go to the quick menu, which is on the right of your profile icon all the way on the left of the menu bar at the lower part of your vision. If you cannot see your menu, put your right palm towards you, and press your index finger to your thumb.



- 4.
5. Once the quick menu opens, press Cast. Then choose Cast to Browser.
6. On your computer, open <https://horizon.meta.com/casting/>
7. Ask one of the TAs to log in into the meta account linked to your device.
8. Once logged in, the VR should start casting, and a red dot will appear in the VR headset.
9. Now you can open the application and work together